

Servicio auth

Registra usuarios y emite los tokens de sesión. Es el único que crea tokens; los demás servicios solo los validan.

EN EL REPOSITORIO

```
services/auth/
├─ serverless.yml
├─ functions.js
├─ requirements.txt + bcrypt
├─ src/handlers/
│   ├─ register/ + integration
│   ├─ login/ + integration
│   ├─ me/
│   └─ logout/
```

Cada función: handler.py · function.yml · test.py (3 casos). Sin utils/ ni data/.

EN AWS

Stack `cdts-<stage>-auth`

API Gateway · 4 rutas /auth/*
CORS abierto, una ruta por función

4 Lambdas
`cdts-<stage>-auth-register · -login · -me · -logout`

IAM mínimo
solo puede leer /cdts/<stage>/db en SSM; sin buckets

SUS 4 FUNCIONES · TODAS SON API (HANDLERS/)

POST /auth/register register/ test.py · integration.py	Valida email y clave (10+ caracteres, letra y número), guarda al usuario con bcrypt y emite su primer token. valid_email password_reason hash_password Users.create issue_token	201 creado + token 400 datos inválidos 409 email ya existe
POST /auth/login login/ test.py · integration.py	Busca al usuario por email, compara la clave con bcrypt y emite un token nuevo. Users.get_by_email verify_password issue_token	200 usuario + token 400 faltan campos 401 credenciales inválidas
GET /auth/me me/ test.py	Devuelve el perfil del dueño del token, sin el password_hash. @require_auth Users.public_dict	200 perfil 401 token ausente o vencido
POST /auth/logout logout/ test.py	Revoca solo el token de esta petición (marca revoked_at); las otras sesiones siguen vivas. @require_auth BearerTokens.revoke	200 revocado 401 token ausente o vencido

Cómo lo usan los demás

files forms processes

Importan @require_auth de libs/utils/auth.py y validan el token leyendo bearer_tokens y users directo en la BD. Nunca llaman a /auth por HTTP.

EN LA BASE DE DATOS

users una fila por cuenta

id	uuid
email	único, en minúsculas
password_hash	bcrypt, costo 12
full_name	nombre para mostrar
created_at	y updated_at

la escribe: register
la leen: login, me y @require_auth

bearer_tokens una fila por sesión

user_id	dueño, borra en cascada
token_hash	SHA-256 del token, único
expires_at	emisión + 24 horas
revoked_at	vacío hasta el logout

la escriben: register, login y logout
la leen: @require_auth en 4 servicios

CICLO DE VIDA DE UN TOKEN

- 1 Se emite en register o login; el cliente lo recibe una vez.
- 2 Se usa en cada petición: Authorization: Bearer <token>.
- 3 Termina con logout (revoked_at) o a las 24 horas.

El token se entrega una sola vez

En la BD solo queda su hash SHA-256; si se filtra la tabla, no sirve.

Mismo error para email y clave

invalid_credentials no revela si el usuario existe.

bcrypt solo vive aquí

passwords.py lo aísla: es el único servicio que lo instala.

Validar no requiere llamar a auth

Cualquier servicio usa @require_auth contra la BD.